SISTEMAS DISTRIBUÍDOS / PROJETO ACADÊMICO

DockerCode

Execução de código com serviços distribuídos e sandboxes Docker

---

02

APIs de submissão

02

Workers concorrentes

04

linguagens

01

fila RabbitMQ

HTTP + AMQP + PostgreSQL + Docker Engine

O problema e a solução

PROBLEMA

Executar código arbitrário dentro da própria API aumenta o risco e mistura atendimento HTTP com processamento potencialmente lento.

SOLUÇÃO IMPLEMENTADA

A API registra e enfileira Jobs. Workers independentes buscam o código persistido, executam cada tentativa em um container temporário e devolvem o resultado.

DECISÃO ARQUITETURAL

Ingestão ≠ execução • processamento assíncrono • isolamento efêmero

Fonte: README.md; serviços/SubmissionApi; ExecutionWorker

Como o projeto está organizado

SRC/

React, Vite, Monaco Editor

Interface, editor e polling

SERVICES/

C# / .NET 8

Submission API • Result Service • Worker

SCRIPTS/

Node.js (MJS)

Smoke test + load test

INFRA/NGINX/

Nginx

Front-end + balanceamento least_conn

SANDBOX-IMAGES/

4 imagens Docker

Python • Node • Java • C++

DOCKER-COMPOSE.YML

Docker Compose

Serviços, banco, broker e volumes

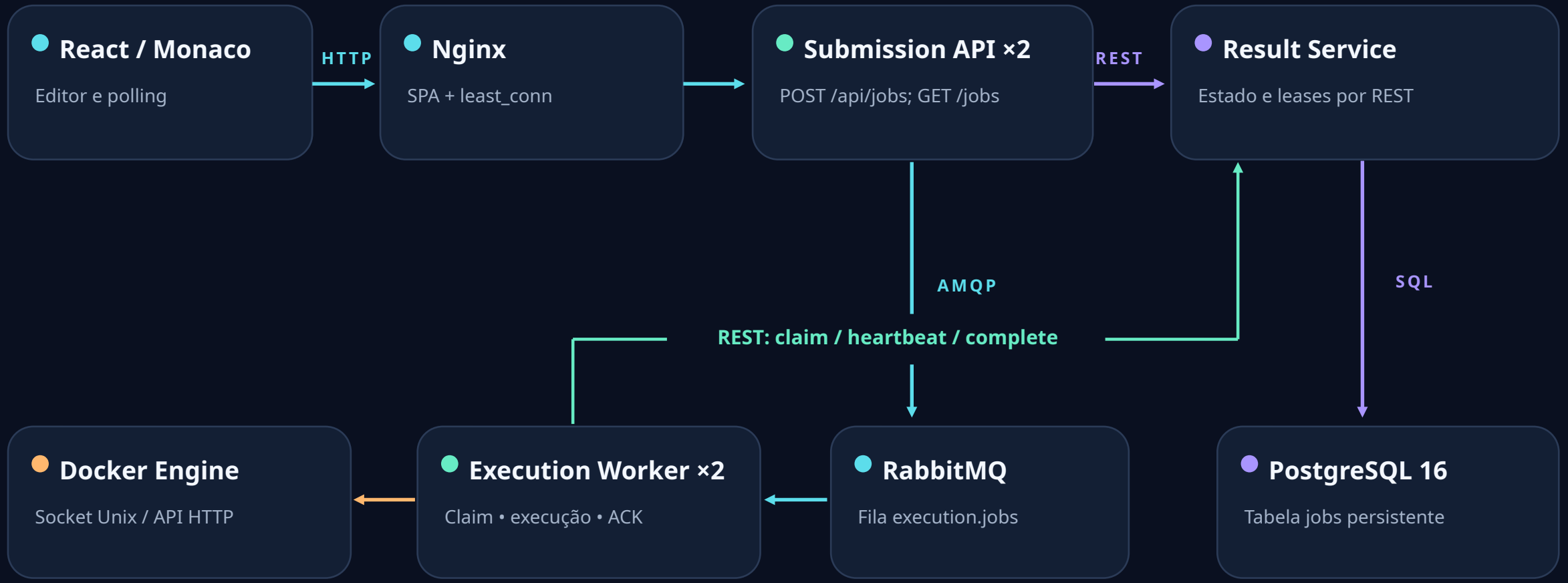
PostgreSQL 16

RabbitMQ 3.13

Docker Engine

Fonte: estrutura real do ZIP; package.json; Dockerfiles

Arquitetura distribuída real



FILA DURÁVEL + CONSUMIDORES COM PREFETCH=1

Fonte: docker-compose.yml; nginx/default.conf; JobConsumer.cs

O caminho de um Job, sem atalhos

01
Run no editor
language + código ativo + stdin

02
POST /api/jobs
Nginx → uma das duas APIs

03
Persistir QUEUED
Result Service → PostgreSQL

04
Publicar no broker
RabbitMQ recebe ID + correlação

05
Worker faz claim
ID → REST recupera código + stdin

06
Criar tmpfs + Docker
Volume e container efêmeros

07
Transferir TAR
PUT /containers/{id}/archive

08
Executar e capturar
wait + stdout/stderr + exit code

09
Persistir + ACK
Concluir com lease válido; limpeza

10
Polling no React
GET /api/jobs/{id} → Console

Importante: RabbitMQ não transporta código-fonte; o Worker busca o código no Result Service.

Fonte: src/App.jsx; SubmissionApi; ResultService; Worker; DockerEngineClient

Sandbox efêmero: limites e isolamento

256 MiB

memória / sandbox

1 CPU

NanoCpus = 10^9

10 s

timeout padrão

64 PIDs

limite de processos

network none

rede desativada

rootfs RO

escrita em tmpfs

LIMITE DE SEGURANÇA

O Worker tem acesso privilegiado ao Docker socket. Isolamento adequado à demonstração local, não prova segurança de produção.

Fonte: DockerEngineClient.cs; sandbox-images/; docker-compose.yml

Demonstração ao vivo: um Job real

01 NAVEGADOR / LOCALHOST:8080

```
nome = input()
print("Olá,", nome)
print("Executado em Docker")
```

I/O → Sistemas Distribuídos

Run

02 TERMINAL / WORKERS

- **POST → 202 / QUEUED**
  A API registra e publica o Job
- **Worker 1 ou Worker 2**
  Obtém claim e cria container
- **SUCCEEDED + stdout**
  Polling a cada 750 ms

Fonte: src/App.jsx; src/services/api.js; README.md

Concorrência e testes: o que o código mede

Smoke: 10 casos + 4 Jobs paralelos = 14

FASE 1

Baseline

8 Jobs

c = 1

FASE 2

Nominal

10 Jobs

c = 2

FASE 3

Burst

24 Jobs

c = 24

FASE 4

Hostil

10 Jobs

c = 10

FASE 5 — AUDITORIA (0 JOBS)

Health da API + busca por containers e volumes órfãos

52 Jobs no total • 2 Workers com prefetch=1 • carga completa: preparar antes da aula

Fonte: scripts/smoke-test.mjs; scripts/load-test.mjs

Resultados: medir antes de concluir

Ingestão HTTP	**POST → 202 Accepted** Tempo de aceite do Job
Espera até claim	**startedAt − createdAt** Inclui publicação e espera; não é Rabbit puro
Execução (intervalo)	**finishedAt − startedAt** Inclui Worker, Docker e persistência
Turnaround	**finishedAt − createdAt** Tempo completo do lado servidor

SEM NÚMEROS INVENTADOS

P50 / P90
P95 / P99

Percentis por cenário;
throughput em Jobs/s.

VALIDATION.md comprova validações anteriores, mas não contém medições da carga completa.

Fonte: cálculos reais em scripts/load-test.mjs; VALIDATION.md

Conclusão: o que foi entregue

IMPLEMENTADO

2 APIs, 2 Workers, Nginx least_conn, RabbitMQ durável, PostgreSQL, leases e sandboxes efêmeros com controles de recursos.

PRÓXIMOS PASSOS

Autenticação, observabilidade, isolamento mais forte e ensaios controlados de falha, throughput e distribuição em múltiplos hosts.

A PRINCIPAL IDEIA

Receber, enfileirar, executar e responder — em serviços independentes.

Fonte: código analisado + seções VALIDADO / NÃO VALIDADO de VALIDATION.md